

N-Party Cyclic Atomic Swap Protocol Specification

A protocol description

April 21, 2026

Contents

1	Setting	2
1.1	Cycle	2
1.2	Transfers	2
2	Core Data	3
2.1	Aggregate Adaptor Secret	3
2.2	Transactions Per Transfer	3
2.3	Locked Account	3
2.4	Refund Windows	3
3	Protocol Phases	4
4	Config Phase	5
5	Setup Phase	5
5.1	Preliminary — Adaptor Point Broadcast	6
5.2	Round 1 — Public Key Exchange (Key1)	7
5.3	Round 2 — Nonce Exchange (Key2)	7
5.4	Round 3 — Pre-signature Exchange (Key3)	7
5.5	Round 4 — Input Transaction Signing (SignInTx)	8
6	Lock Phase	8
7	Claim Phase	9
7.1	Leader's Role (P_0)	9
7.2	Non-Leader Roles ($P_j, j \neq 0$)	9
8	Refund Phase	10
9	Refund Window Safety Argument	10
10	Per-Party State	11
11	Example Trace: $N = 3$, Leader = A	11
12	Example Trace: $N = 3$, Refund Path	11
13	Scope and Open Issues	14

1 Setting

A finite set of N parties $P = \{P_0, P_1, \dots, P_{N-1}\}$. The party P_0 is the designated **leader**.

1.1 Cycle

The parties are arranged in a directed cycle:

$$P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_{N-1} \rightarrow P_0.$$

Each directed edge defines one **transfer**.

1.2 Transfers

Transfer _{i} is the transfer from P_i to $P_{(i+1) \bmod N}$, for $i = 0, \dots, N - 1$.

Roles. Because the swap is cyclic, *every* party plays all three of the following roles simultaneously:

- **Depositor** for transfer _{i} : P_i locks funds on-chain and, in Setup, drives the transaction-signing round.
- **Claimant** for transfer _{$i-1$} : P_i drives the key-exchange rounds of Setup and later claims the locked funds.
- **Co-signer** for all N transfers: P_i contributes its public key, nonce, and partial pre-signature to the multi-sig lock of every transfer's locked account.

The terms *sender* and *receiver* are used below only as shorthand for the depositor and claimant roles *within a specific transfer*. They do not denote distinct categories of party.

The leader P_0 is the claimant in transfer _{$N-1$} (the transfer from P_{N-1} to P_0). Until the Claim phase, all parties execute identical protocol steps, differing only in which transfer index they are depositing into and claiming from. **The only asymmetry in the protocol is in the Claim phase** (Section 7), where the leader's behaviour diverges from that of all other parties.

Figure 1 shows the structure for $N = 3$.

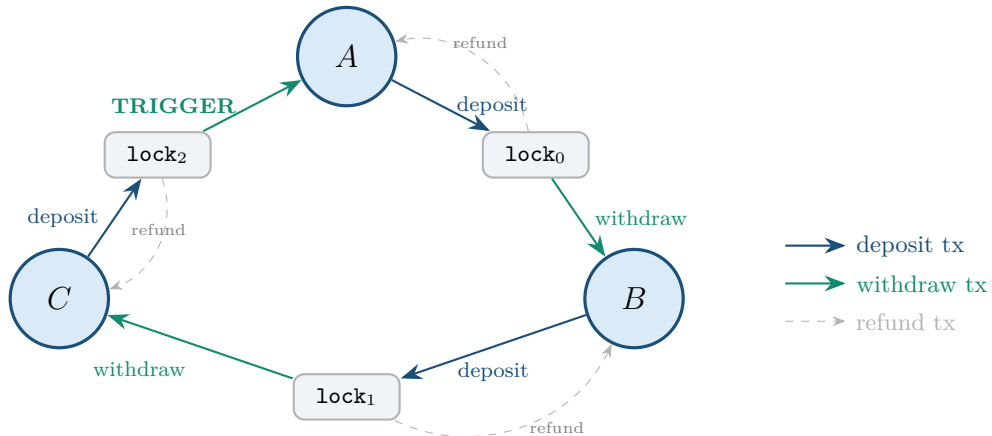


Figure 1: Three-party cyclic swap ($N = 3$, leader = A). Blue solid arrows: deposit path. Teal solid arrows: withdraw (claim) path. The **TRIGGER** is the first withdraw broadcast, by the leader A claiming C 's locked funds. Dashed arrows: refund path (time-locked, unhappy path only).

2 Core Data

2.1 Aggregate Adaptor Secret

Each party P_i holds a private **adaptor secret** t_i . The corresponding public **adaptor point** is $T_i = t_i \cdot G$, where G is the curve generator. T_i is broadcast to all parties at the start of the Setup phase. The **aggregate adaptor point** T_{agg} is the sum of all individual adaptor points. The corresponding **aggregate adaptor secret** is:

$$t_{\text{agg}} = t_0 + t_1 + \dots + t_{N-1}.$$

This single secret is used to complete the withdraw transaction for *every* transfer. It is distributed: no strict subset of parties can compute it unilaterally.

2.2 Transactions Per Transfer

For each transfer _{i} three transactions are constructed during Setup:

Transaction	Spends	Produces	Authorisation
Deposit tx	P_i 's own funds	Locked account for transfer _{i}	P_i 's signature alone
Withdraw tx	Locked account	$P_{(i+1) \bmod N}$'s address	Aggregate multi-sig + t_{agg}
Refund tx	Locked account	P_i 's address	Aggregate multi-sig + block $\geq W_i$

The **withdraw tx** is constructed as a *pre-signature* during Setup and completed only during the Claim phase when t_{agg} becomes known. The **refund tx** is fully co-signed during Setup but is time-locked and cannot be broadcast before block height W_i .

2.3 Locked Account

Each transfer's locked account is an N -of- N multi-signature account. Its unlocking key is an aggregated public key derived from all N parties' individual public keys. Spending it requires a single aggregated Schnorr signature committing all N parties.

Mutual exclusion assumption. For each transfer's locked account, the underlying chain guarantees that at most one of the withdraw tx and the refund tx can be confirmed. In UTXO-based chains this holds structurally (spending a UTXO consumes it, invalidating any other transaction that references it); other chain models must enforce it explicitly.

2.4 Refund Windows

Each transfer _{i} has an associated **refund window open height** W_i : the minimum block height of a chosen blockchain at which the refund tx for transfer _{i} becomes valid.

Staggered window invariant (cyclic case):

$$W_0 > W_1 > W_2 > \dots > W_{N-1}.$$

The leader P_0 's refund window (for transfer₀, the deposit made by P_0) opens **last**. P_{N-1} 's refund window (for the deposit into the leader) opens **first**.

Minimum gap requirement. Let Δ be the **claim propagation time**: the maximum number of blocks required for a party to (1) observe a broadcast transaction at the required finality depth, (2) construct and sign the corresponding withdraw tx, and (3) have it confirmed

to the required finality depth. The staggered ordering alone is not sufficient for safety; the gaps must satisfy:

$$W_i - W_{i+1} \geq \Delta \quad \text{for all } i = 0, \dots, N-2.$$

Δ is a protocol parameter that must be fixed before the Config phase and depends on the confirmation-depth policies of the relevant blockchains. This value **must be chosen conservatively** as the security of the protocol depends entirely on it.

Security note. If $W_i - W_{i+1} < \Delta$, a malicious leader (the only party that controls the trigger broadcast) can allow W_{i+1} to expire before $P_{(i+1) \bmod N}$ can confirm its withdraw tx. $P_{(i+1) \bmod N}$ then claims a refund, but the adversary also claims $P_{(i+1) \bmod N}$'s deposit via the withdraw tx: that party loses their deposit and receives nothing, breaking atomicity.

Griefing opportunity cost asymmetry. Because refund windows are staggered, an abort imposes unequal waiting costs on different parties. If any party aborts after the Lock phase, every party eventually recovers their principal via the refund path; no principal is lost. However, a party with a small W_j (refund available early) that withholds their adaptor secret forces the leader, who holds the largest window W_0 , to keep funds locked for the longest period. The aborting party bears a shorter opportunity cost than the leader does. This asymmetry is a liveness concern, not an atomicity violation.

Optionality. The temporal gap between the Lock phase (when all funds are committed) and the leader's decision to broadcast the trigger creates a free option for the leader. During this gap the leader can observe price movements and choose to complete the swap (if prices moved favourably) or let it expire via refund (if prices moved against them). The non-leader parties are locked in and cannot exit. Han et al. [1] prove that this is structurally equivalent to a premium-free American call option granted to the leader at the non-leaders' expense, and estimate the fair option premium at 2–3% of the swap value for typical cryptocurrency volatility. The protocol does not require the leader to broadcast the trigger within any deadline, nor does it impose a premium to compensate non-leaders for this optionality. The refund path ensures no principal is lost, but expected value is transferred from non-leaders to the leader through the option itself. This is not specific to this protocol: it is a structural property of all atomic swap protocols (HTLC or adaptor-signature-based) in which one party commits last.

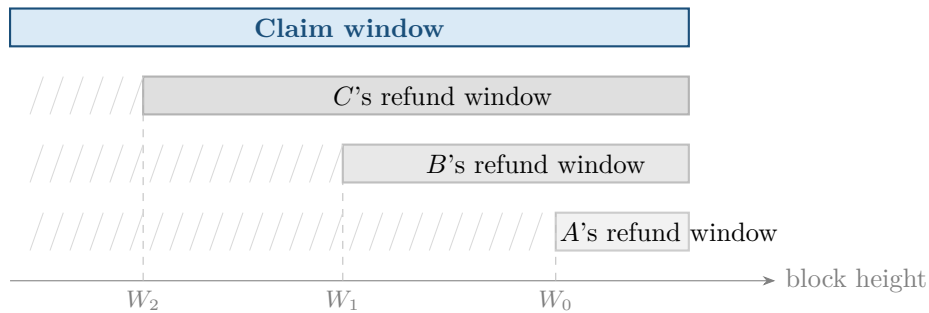


Figure 2: Staggered refund windows for $N = 3$. $W_2 < W_1 < W_0$. The claim window is the period during which withdraw transactions may be broadcast. Hatched regions are the blocks before a party's refund window is active.

3 Protocol Phases

Each transfer independently progresses through five phases. Phases for different transfers run concurrently but are synchronised at two global barriers (Lock and Claim, see Sections 6 and 7).

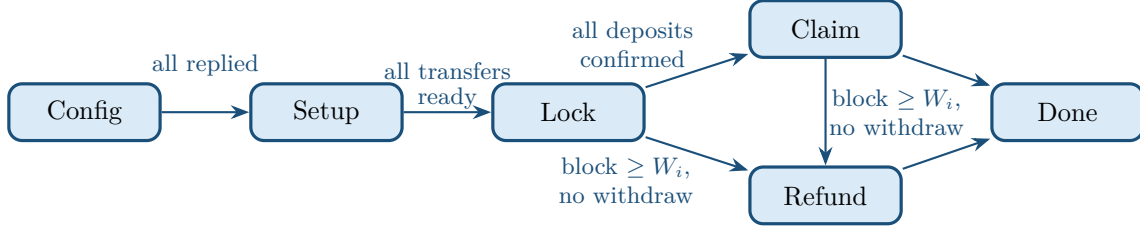


Figure 3: Phase state machine for a single transfer. The Lock $\rightarrow$ Claim transition fires when the party has observed all N deposits confirmed on-chain. The Lock $\rightarrow$ Refund and Claim $\rightarrow$ Refund transitions fire when the refund window opens and the withdraw tx has not yet been broadcast.

Local synchronisation conditions:

- A party enters **Lock** only after it has completed all four Setup rounds for every transfer.
- A party enters **Claim** only after it has observed all N deposit txs confirmed on-chain at the required finality depth.

4 Config Phase

Goal. Establish the swap parameters, have each party generate its private adaptor secret, and confirm that all N parties are willing to proceed before any cryptographic material is exchanged between parties.

Config has two parts: a one-time global step and a per-transfer consent step.

Part 1 — Swap description (once per swap). The swap initiator distributes the **swap description** to all N parties: a unique swap identifier, the ordered party list $[P_0, \dots, P_{N-1}]$, the leader's identity, and the proposed refund window heights $W_0, \dots, W_{N-1}$. Each party P_i records *swap_id*, *parties*, and *leader* = P_0 , and generates its private **adaptor secret** t_i uniformly at random.

Part 2 — Per-transfer consent. For transfer _{i} , $P_{(i+1) \bmod N}$ proposes the swap description to all N parties; each party either accepts or withholds consent. $P_{(i+1) \bmod N}$ proceeds to Setup only after receiving consent from all N parties.

Pre-condition (for a party to consent):

- The party has received the swap description: the transfer cycle, the complete ordered party list, the leader's identity, and the proposed refund window heights $W_0, \dots, W_{N-1}$.
- The proposed windows satisfy the staggered invariant $W_0 > W_1 > \dots > W_{N-1}$ with gaps $W_i - W_{i+1} \geq \Delta$.

Post-condition: Every party holds the swap description and its own private adaptor secret t_i . All N parties have agreed on the swap description and $P_{(i+1) \bmod N}$ may proceed to Setup for transfer _{i} .

5 Setup Phase

Goal. Construct and mutually verify all three transactions for every transfer so that:

1. Everyone can independently compute T_{agg} .

2. For each transfer _{i} , $P_{(i+1) \bmod N}$ holds a verified pre-signature for the withdraw tx (completable once t_{agg} is known, and not before).
3. For each transfer _{i} , P_i holds a fully co-signed, time-locked refund tx.
4. All parties have verified that every refund window W_i is correctly set.

Setup begins with a one-time preliminary step in which every party broadcasts its individual adaptor point T_i to all other parties. This happens once per swap, before any per-transfer rounds begin.

The per-transfer rounds then run once for each transfer and every party participates in every instance. For transfer _{i} , the four rounds are structured as follows: Rounds 1–3 are initiated by $P_{(i+1) \bmod N}$ in its claimant role; Round 4 is initiated by P_i in its depositor role. All N parties respond in every round for every transfer. Since there are N transfers, each party initiates Rounds 1–3 exactly once, initiates Round 4 exactly once, and responds in all four rounds N times. Figure 4 shows the per-transfer sub-protocol.

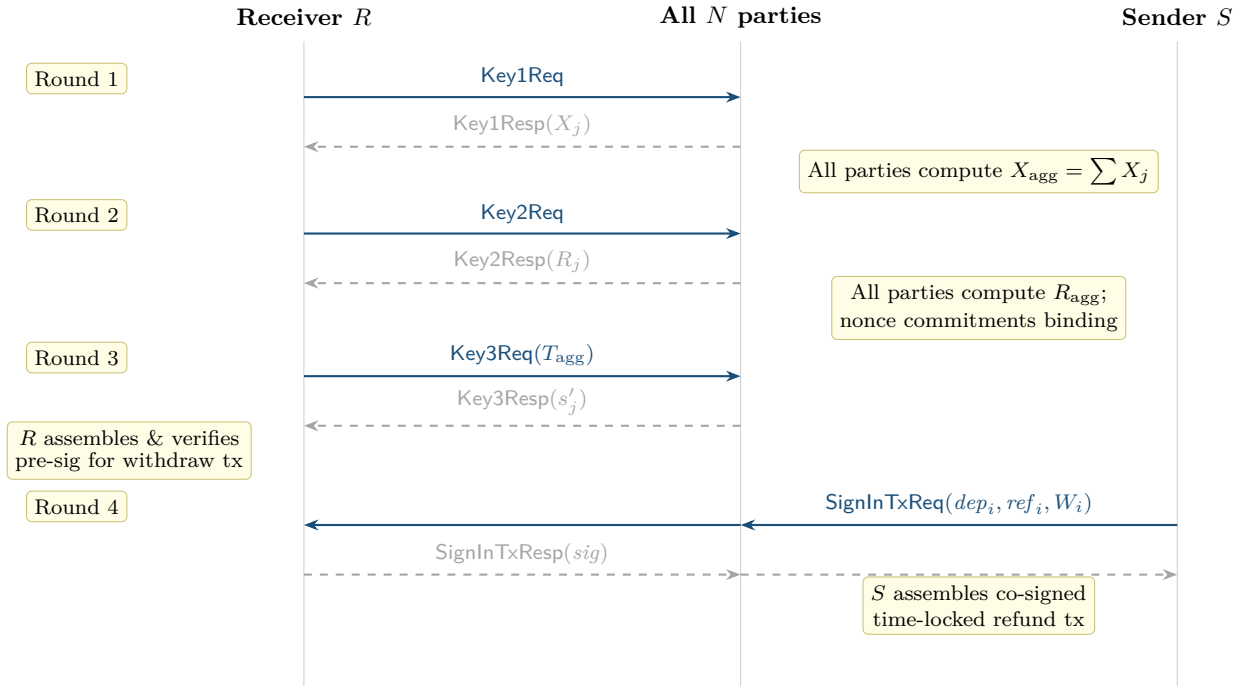


Figure 4: Setup per-transfer sub-protocol sequence. The preliminary adaptor point broadcast (once per swap, before these rounds) is described in §5.1. Rounds 1–3: receiver-driven key and pre-signature exchange. Round 4: sender-driven transaction signing. Each **Key k Req** is broadcast to all N parties; responses return to the initiator.

5.1 Preliminary — Adaptor Point Broadcast

This step runs **once per swap**, before any per-transfer rounds begin.

Every party P_j computes its **adaptor point** $T_j = t_j \cdot G$ from its private adaptor secret t_j , and broadcasts T_j to all N parties. All parties store the full set $\{T_0, T_1, \dots, T_{N-1}\}$ for later use in Round 3.

Pre-condition: Config complete for all transfers.

Post-condition: Every party holds the individual adaptor points T_j for all j and can independently compute the aggregate adaptor point $T_{\text{agg}} = T_0 + T_1 + \dots + T_{N-1}$. If any

party withholds or sends a malformed T_j , the discrepancy is detected no later than Round 3, when co-signers independently verify T_{agg} against the individual adaptor points.

5.2 Round 1 — Public Key Exchange (Key1)

Initiator for transfer_i: $P_{(i+1) \bmod N}$ broadcasts Key1Req to all N parties.

Every party P_j responds with its **public key** X_j . All parties compute the **aggregated public key** $X_{\text{agg}} = X_0 + X_1 + \dots + X_{N-1}$, which determines the locked account address for transfer_i.

Pre-condition: Preliminary step complete; Config complete for transfer_i.

Post-condition: All parties agree on X_{agg} and hence on the locked account address for transfer_i.

5.3 Round 2 — Nonce Exchange (Key2)

Initiator for transfer_i: $P_{(i+1) \bmod N}$ broadcasts Key2Req to all N parties.

Every party P_j responds with its **public nonce** R_j . All parties compute the **aggregate nonce** $R_{\text{agg}} = R_0 + \dots + R_{N-1}$.

Pre-condition: Round 1 complete for transfer_i.

Post-condition: All parties are committed to a specific nonce for partial signature computation for transfer_i. Changing a nonce after this point invalidates all collected partial signatures, making the commitment binding by construction.

5.4 Round 3 — Pre-signature Exchange (Key3)

Initiator for transfer_i: $P_{(i+1) \bmod N}$ broadcasts Key3Req together with the claimed aggregate adaptor point T_{agg} to all N parties.

Before computing its partial pre-signature, each responding party **must verify**:

- $T_{\text{agg}} = T_0 + T_1 + \dots + T_{N-1}$, computed independently from the individual adaptor points T_j broadcast in the Preliminary step of Setup.
- The withdraw tx being pre-signed spends the correct locked account (the address determined by X_{agg} in Round 1) and credits $P_{(i+1) \bmod N}$'s address for the agreed amount.

Security note. If co-signers do not verify T_{agg} , the initiating party can substitute $T'_{\text{agg}} = T_{\text{agg}} + \delta \cdot G$ for an arbitrary scalar δ . The resulting pre-signature requires $t_{\text{agg}} + \delta$ to complete. That value is extractable on-chain after the trigger, but it is not t_{agg} ; all other parties' withdraw transactions, which require t_{agg} , become permanently invalid, and the initiating party alone can claim.

Every responding party P_j computes its **partial pre-signature** s'_j over the (verified) withdraw tx for transfer_i and sends it to $P_{(i+1) \bmod N}$. $P_{(i+1) \bmod N}$ assembles the **aggregate pre-signature** from all N responses and verifies it against the (verified) T_{agg} .

Pre-condition: Round 2 complete; all parties hold the individual adaptor points T_j for all j (broadcast in the Preliminary step).

Post-condition: $P_{(i+1) \bmod N}$ holds a verified aggregate pre-signature for the withdraw tx of transfer_i. This pre-signature is not a valid signature; it becomes one if and only if t_{agg} is applied to it. No party can complete this withdraw tx without knowing t_{agg} .

5.5 Round 4 — Input Transaction Signing (SignInTx)

Initiator for transfer_i: P_i broadcasts **SignInTxReq** containing the proposed **deposit transaction** dep_i , the proposed **refund transaction** ref_i , and the claimed refund window height W_i .

Before co-signing, each party **must verify all of the following**:

- The deposit tx sends funds to the locked account address determined by X_{agg} in Round 1 (not a substituted address).
- The deposit tx amount matches the agreed transfer amount for transfer_i.
- The refund tx spends the correct locked account for transfer_i.
- The refund tx credits P_i 's address.
- The refund tx time-lock is exactly W_i .
- W_i is consistent with the window heights agreed in the Config phase and satisfies the staggered invariant ($W_{i-1} > W_i > W_{i+1}$, with boundary conditions at $i = 0$ and $i = N - 1$).

Security note. Without these checks, a malicious depositor can present a deposit tx that sends to an attacker-controlled address, or a refund tx with a shortened time-lock, and obtain co-signatures on it from all other parties. A mis-addressed deposit tx allows the attacker to claim the locked funds directly; a shortened refund lock allows a refund before the claim window closes, in both cases forfeiting the rightful claimant's output.

A party that fails any of the above checks withholds its **SignInTxResp**, causing Setup to stall; that party waits for its refund window to open and broadcasts its refund tx.

P_i assembles the fully co-signed refund tx from all N responses.

Pre-condition: Round 3 complete for transfer_i.

Post-condition (P_i): Holds the deposit tx (ready to broadcast) and a fully co-signed time-locked refund tx valid at block height $\geq W_i$.

Post-condition (all parties): Have verified the deposit tx destination, the refund tx recipient and time-lock, and the refund window W_i against the agreed protocol parameters.

6 Lock Phase

Goal. All senders deposit their funds into their respective locked accounts on-chain.

Pre-condition: The party has completed all four Setup rounds for all N transfers and has verified pre-signatures and refund windows for every transfer. A party does not lock funds unless satisfied that it can either claim its output or obtain a refund.

Actions:

1. Every party P_i broadcasts its deposit tx for transfer_i.
2. All parties monitor all blockchains and wait for finality of all N deposit txs.

Post-condition: For each transfer_i, the locked account holds P_i 's funds. The only ways to spend those funds are: (a) the withdraw tx (requiring t_{agg}), or (b) the refund tx (after block W_i , with the already co-signed multi-sig).

7 Claim Phase

Goal. Every party claims the funds locked for the transfer it is receiving.

Pre-condition: The party has observed all N deposit txs confirmed on-chain at the required finality depth.

This is the only phase where parties are treated differently. Up to this point the protocol has been fully symmetric: every party has acted as co-signer for all N transfers, driven Rounds 1–3 for exactly one transfer, and driven Round 4 for exactly one transfer. The Claim phase breaks this symmetry solely through the role of the leader P_0 , whose behaviour diverges from that of all other parties as described below.

7.1 Leader's Role (P_0)

1. P_0 solicits adaptor secrets t_j from each non-leader party P_j ($j = 1, \dots, N-1$) over a private off-chain channel.
2. Each non-leader party P_j sends t_j to P_0 **only after** independently confirming that all N deposits are finalised on-chain.
3. P_0 receives t_j from each P_j and verifies it against the corresponding public adaptor point T_j .
4. P_0 computes $t_{\text{agg}} = t_0 + t_1 + \dots + t_{N-1}$, completes the pre-signature for the withdraw tx of transfer $_{N-1}$, and broadcasts it — the **trigger transaction**.

Deviation — withheld or malformed secret. If any P_j withholds t_j , or the received value does not verify against T_j , P_0 cannot assemble t_{agg} and does not broadcast the trigger. All parties fall through to the **Refund phase**: each party P_i waits until block height $\geq W_i$ and broadcasts their refund tx. No party loses funds because refund txs were fully co-signed during Setup.

Critical constraint. P_0 must **never** share t_0 off-chain. P_0 's adaptor secret is disclosed only through the on-chain broadcast of the trigger tx. Were P_0 to share t_0 off-chain, the remaining $N-1$ parties could reconstruct t_{agg} independently and time the trigger broadcast to expire just before P_0 's refund window W_0 , cheating P_0 .

Post-condition: Trigger tx on-chain; t_{agg} publicly extractable from its signature.

7.2 Non-Leader Roles ($P_j, j \neq 0$)

1. P_j observes a completed withdraw tx on-chain (typically the trigger) and derives t_{agg} from its signature. Any completed withdraw tx reveals t_{agg} , since every withdraw tx is authorised by the same aggregate adaptor secret.
2. P_j completes the pre-signature for the withdraw tx of transfer $_{j-1}$ (the transfer from P_{j-1} to P_j) using t_{agg} and broadcasts it.

Deviation — trigger never arrives. If P_j does not observe the trigger tx by the time block height W_j is reached, P_j concludes that the Claim phase has failed and transitions to the **Refund phase**: P_j broadcasts their refund tx for transfer $_j$.

Post-condition (happy path): All N withdraw txs broadcast; each party has claimed their output.

8 Refund Phase

Goal. Any party whose outgoing withdraw tx has not been broadcast reclaims their deposit once their refund window opens.

This phase is entered whenever the Claim phase fails to complete — whether because a secret was withheld, a secret was malformed, or the trigger was never observed before the refund window opened. No distinction is made between these cases: the party acts on the timeout alone.

Pre-condition for P_i to broadcast their refund tx:

- Block height $\geq W_i$.
- The withdraw tx for transfer_i has not been broadcast.

Action. P_i broadcasts the fully co-signed refund tx for transfer_i , returning their deposit.

Liveness obligation. A party must broadcast their refund tx as soon as W_i is reached and their funds are still locked. Delay risks a concurrent withdraw tx completing first, leaving the party with neither output nor refund. Each party must therefore monitor on-chain state continuously once the Lock phase is complete.

Post-condition. P_i 's deposit is returned. The swap has partially or fully failed; no party has lost funds.

9 Refund Window Safety Argument

The staggered window invariant ensures that no coalition of parties can cheat an honest party out of both their deposit and their output, *provided* the minimum gap requirement is met.

Formal invariant. For the cycle $P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_{N-1} \rightarrow P_0$ with P_0 as leader:

$$W_0 > W_1 > W_2 > \dots > W_{N-1}, \quad \text{with} \quad W_i - W_{i+1} \geq \Delta \quad \text{for all } i = 0, \dots, N-2.$$

The claim propagation time Δ is defined in Section 2.4. If this gap condition is not met, an adversary can exploit the timing to simultaneously collect a withdraw and a refund (see Section 2.4).

Intuition (three-party case, $A \rightarrow B \rightarrow C \rightarrow A$):

- **Coalition against B (A and C collude):** A delays the trigger past C 's window W_2 . If C claims a refund, transfer_2 's funds are gone and A has no trigger to broadcast. A 's deposit in transfer_0 is now locked with no claim path; A is worse off than B . B can still refund via transfer_1 .
- **Coalition against C (A and B collude):** A delays the trigger past B 's window W_1 . B claims a refund on transfer_1 . Because $W_2 < W_1$, C has already either claimed or refunded transfer_2 before B 's window opens. No party is left with neither output nor refund.
- **Leader's window last:** W_0 being last gives P_1 the maximum time to observe the trigger and claim before W_0 opens.
- **Partial claim (trigger broadcast, non-leader slow):** A broadcasts the trigger. C fails to broadcast its withdraw tx for transfer_1 before W_1 is reached. The gap $W_0 - W_1 \geq \Delta$ guarantees that B (depositor of transfer_1) cannot refund before C has had Δ blocks to observe the trigger and broadcast. If C is honest and online, it claims in time; if C is offline, B eventually refunds, and C bears the loss of not claiming—but C 's own deposit in

transfer_2 was already claimed by A via the trigger, so C loses its deposit without receiving its output. This is not an atomicity violation by the protocol: C 's liveness obligation (broadcasting the withdraw tx promptly) is a precondition of the happy path.

10 Per-Party State

The complete local state of party P_i throughout the protocol.

Variable	Type	Set in	Description
swap_id	identifier	Config	Unique identifier for this swap
parties	list of PIDs	Config	Ordered party list $[P_0, \dots, P_{N-1}]$
leader	PID	Config	Always P_0
t_i	secret scalar	Config	P_i 's private adaptor secret
T_j (all j)	public point	Setup Prelim	All parties' public adaptor points
$\text{transfer}_i.\text{phase}$	Phase	Runtime	P_i 's sending-role phase
$\text{transfer}_i.\text{deposit_tx}$	transaction	Setup R4	Ready to broadcast in Lock
$\text{transfer}_i.\text{refund_tx}$	signed tx	Setup R4	Time-locked; broadcast in Refund
$\text{transfer}_i.W_i$	block height	Setup R4	Refund window open height
$\text{transfer}_{(i-1) \bmod N}.\text{presig}$	pre-signature	Setup R3	Aggregate pre-sig for withdraw tx
$\text{transfer}_{(i-1) \bmod N}.\text{withdraw_tx}$	transaction	Claim	Completed in Claim using t_{agg}
$\text{all_deposits_confirmed}$	bool	Lock	True when all N deposits finalised

Leader-specific (P_0 only):

Variable	Type	Set in	Description
collected_secrets	PID $\rightarrow$ scalar	Claim	Secrets t_j from P_j ($j \neq 0$)
t_{agg}	scalar	Claim	Computed once all $N-1$ secrets received

11 Example Trace: $N = 3$, Leader = A

Parties: $A = P_0$ (leader), $B = P_1$, $C = P_2$.

Cycle: $A \rightarrow B \rightarrow C \rightarrow A$.

Transfers: $\text{tr}_0 = (A \rightarrow B)$, $\text{tr}_1 = (B \rightarrow C)$, $\text{tr}_2 = (C \rightarrow A)$ (leader's received transfer).

Refund windows: $W_0 > W_1 > W_2$.

The full sequence diagram is shown in Figure 5.

12 Example Trace: $N = 3$, Refund Path

Parties: $A = P_0$ (leader), $B = P_1$, $C = P_2$.

Cycle: $A \rightarrow B \rightarrow C \rightarrow A$.

Scenario: B withholds t_B during Claim. A cannot assemble t_{agg} and does not broadcast the trigger. All parties wait out their respective refund windows and reclaim their deposits.

The full sequence diagram is shown in Figure 6.

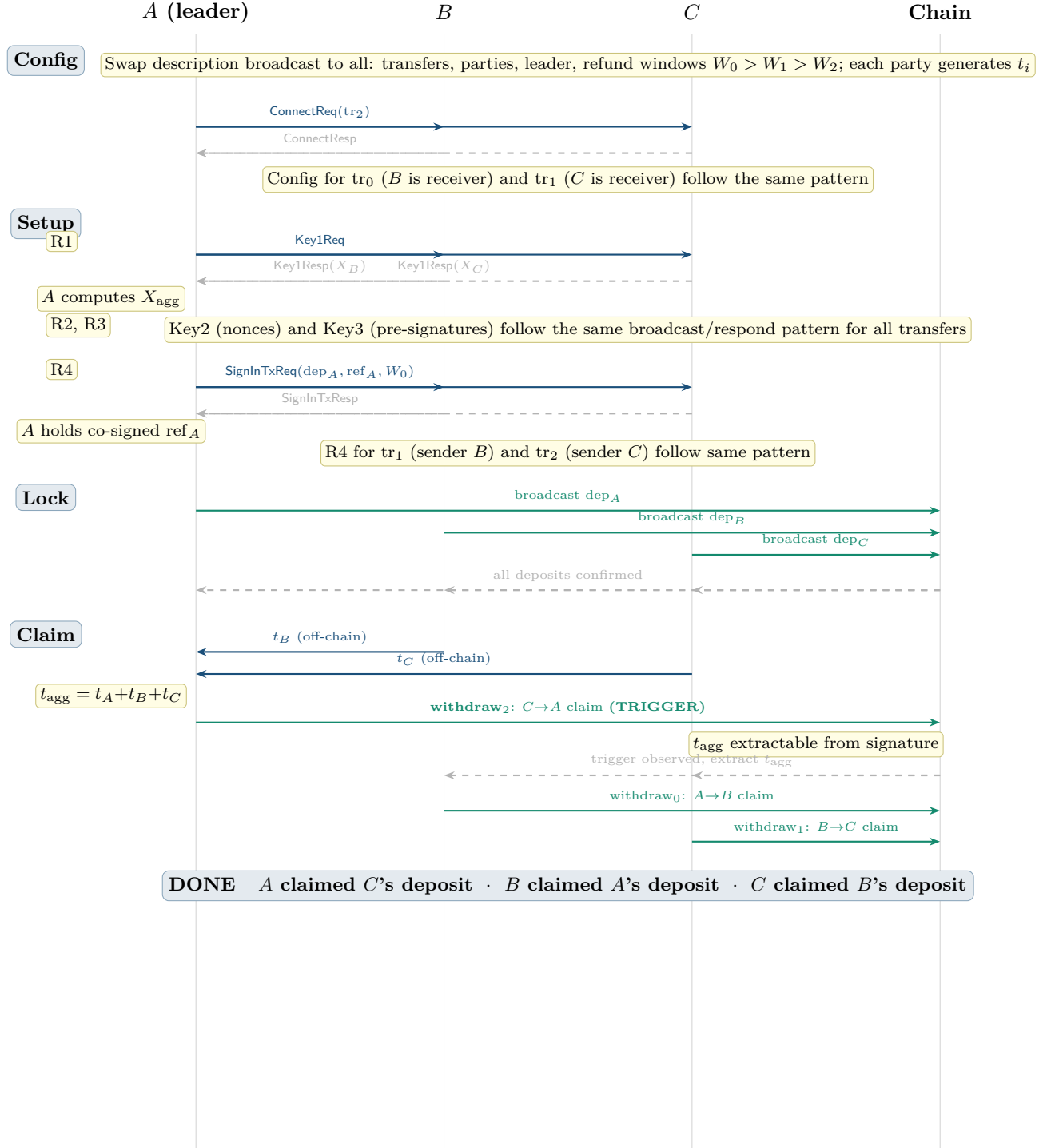


Figure 5: Full happy-path trace for $N = 3$, leader = A. Blue arrows: protocol (off-chain) messages. Teal arrows: on-chain transactions and chain-to-party notifications. Dashed arrows: responses. Config and Setup R1 for tr_0 and tr_1 follow the same pattern as shown for tr_2 . Setup R4 for tr_1 and tr_2 follows the same pattern as shown for tr_0 . Rounds R2 and R3 follow the broadcast/respond pattern noted inline.

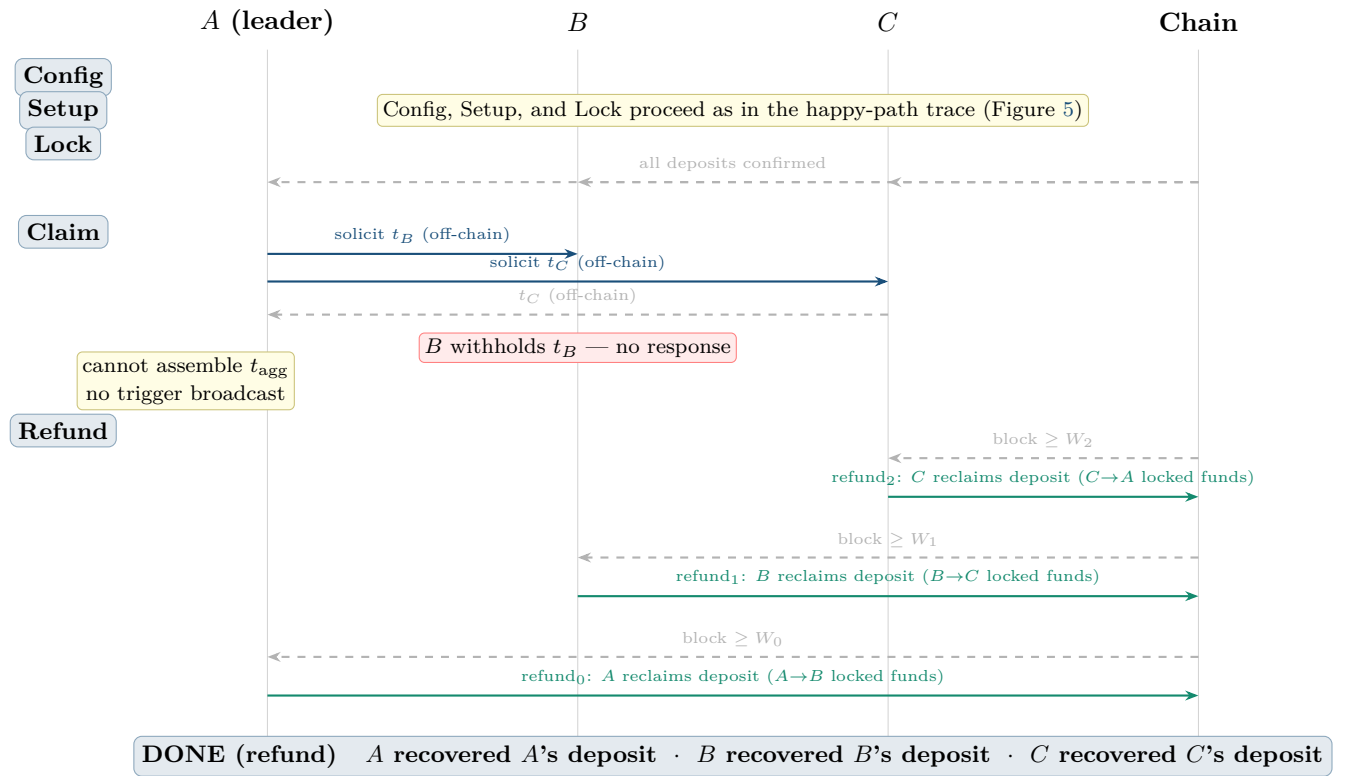


Figure 6: Refund-path trace for $N = 3$, leader = A . B withholds t_B ; A cannot assemble t_{agg} and no trigger is broadcast. C 's refund window W_2 opens first; B 's W_1 next; A 's W_0 last. Each party independently monitors the chain and broadcasts their refund tx as soon as their window opens. No principal is lost. Style conventions as in Figure 5.

13 Scope and Open Issues

- **Non-cyclic swaps** (separate input and output parties): the staggered window invariant does not straightforwardly generalise and remains an open design problem. Out of scope for this specification.
- **Leader election:** the leader is assumed pre-determined. A commit-reveal randomness scheme (hash all nonces, take index $H(\dots) \bmod N$) can be used but is not part of the core protocol.
- **Cross-chain finality:** the definition of “confirmed” for each blockchain is chain-specific and is treated as an oracle input to this protocol.
- **Cryptographic soundness:** the adaptor signature scheme, the MuSig2 multi-signature scheme, and their combination are assumed correct and secure.

References

- [1] Runchao Han, Haoyu Lin, Jiangshan Huang, Zhimei Zhang, and Hong Zhu. On the optionality and fairness of atomic swaps. In *Proceedings of the First ACM Conference on Advances in Financial Technologies (AFT '19)*. ACM, 2019. Also available as IACR ePrint 2019/896.